Credit Card Fraud Detection Using Machine Learning

MLP 2: Mid-Term Report / Progress Report

Team Members: Hongru He, Ramya Ramesh

This progress report builds upon our Problem Statement from MLP1 and the project description in the ML Project Document. It summarizes our data understanding, preprocessing, EDA, initial ML pipeline, baseline models, feature analysis, algorithms tested, challenges, and plan for completion.

1. Project Objective and Problem Statement

This project aims to build a machine learning model for credit card fraud detection. The goal is to classify each transaction as either legitimate or fraudulent based on transaction-related features. This is a binary classification problem with a strong real-world constraint: fraud cases are extremely rare, which makes accurate detection challenging.

Problem Definition & Deployment Context

- **The Task:** Given a credit card transaction, predict whether it is fraudulent (Class=1) or legitimate (Class=0).
- **The Objective:** Minimize false negatives (missed fraud) while controlling false positives (incorrectly flagged transactions).
- **Operational Use:** Operationally, the model is designed to function as an automated, real-time transaction scoring system. High-probability fraud predictions will trigger immediate automated blocks or real-time user verification alerts (e.g., SMS confirmation). Meanwhile, moderate-confidence flags will be routed to a secondary human review queue for fraud analysts to investigate.
- **Business Impact:** In a real-world deployment context, this dual-layered approach directly protects the financial institution's bottom line by reducing direct losses from stolen funds, while simultaneously preserving a seamless customer experience by minimizing the friction of falsely declined cards.

Measurable Success

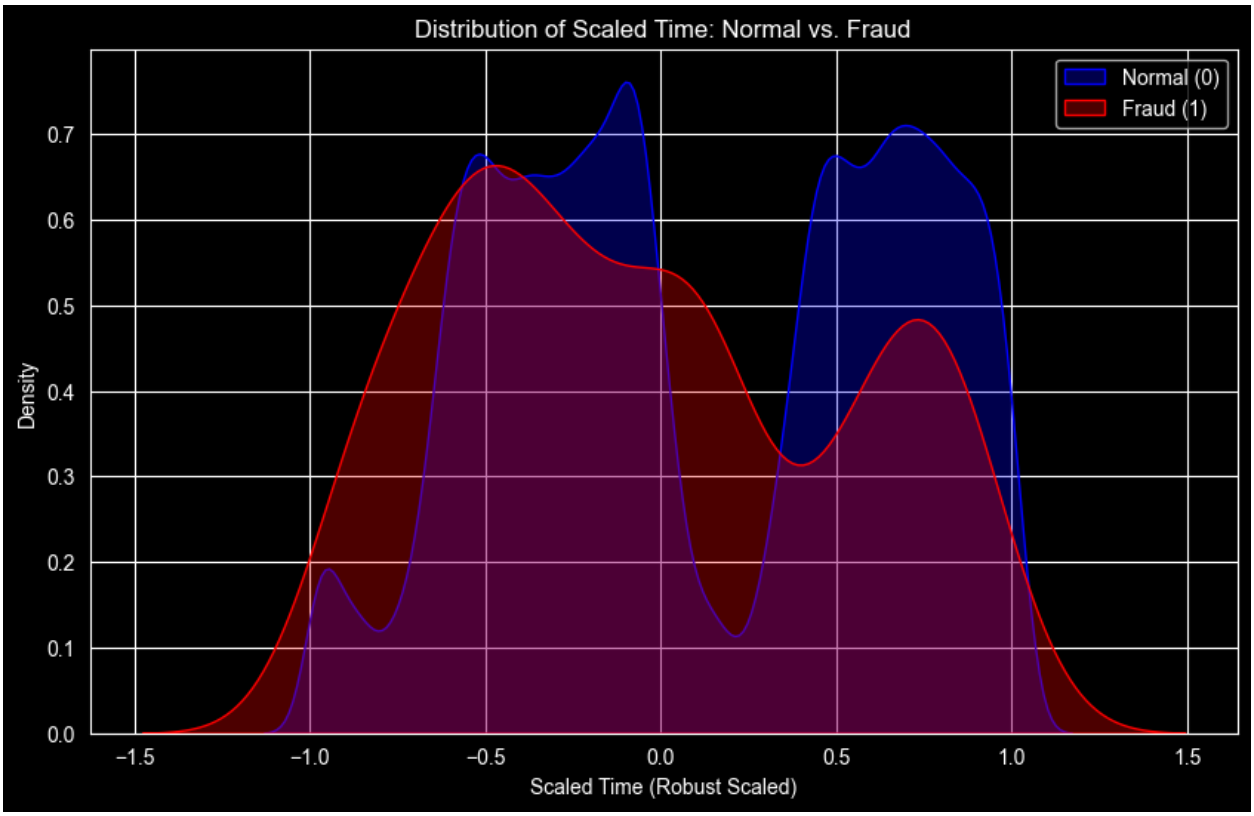
Success will be measured not only by overall accuracy—which is misleading in this context—but by metrics such as precision, recall, F1-score, and ROC-AUC, with particular emphasis on recall for the fraud class due to the severe imbalance in the dataset.

In measurable terms, initial success for the baseline model is defined as achieving a **Recall score exceeding 85%** (ensuring the vast majority of fraud is successfully trapped). Ultimate project success in the optimization phase will be defined by maximizing the **F1-score and PR-AUC** (Precision-Recall Area Under Curve), proving that we can maintain that high Recall while raising Precision to a commercially viable level that does not overwhelm the review queue with false alarms.

2. Data Preprocessing and Cleaning

The dataset contains 284,807 transactions and 31 columns, all numeric. The first step involved exploratory data analysis (EDA) and problem clarification, focusing on understanding feature distributions and scales.

- **Amount Feature:** This is a continuous numerical feature representing the transaction amount. It requires scaling or transformation due to skewness. We applied a RobustScaler to handle extreme outliers effectively without distorting the data distribution.
- **Time Feature:** This feature represents the seconds elapsed since the first transaction in the dataset. To evaluate its utility, we scaled it and analyzed its correlation with the target variable. Finding a near-zero correlation and recognizing that a cumulative counter lacks generalizable predictive power for future, real-time transactions, we permanently dropped the Time feature from our pipeline.
  [**Pearson Correlation between Scaled Time and Class: -0.01232**]



Beyond the statistical overlap, there is a fundamental real-world deployment constraint. Time acts as a cumulative counter from an arbitrary starting point. In a live production environment, the model would not have a meaningful baseline "first transaction" to measure against. Retaining this feature risks the model learning non-generalizable, spurious patterns. Consequently, it is permanently dropped from the logistic regression pipeline.

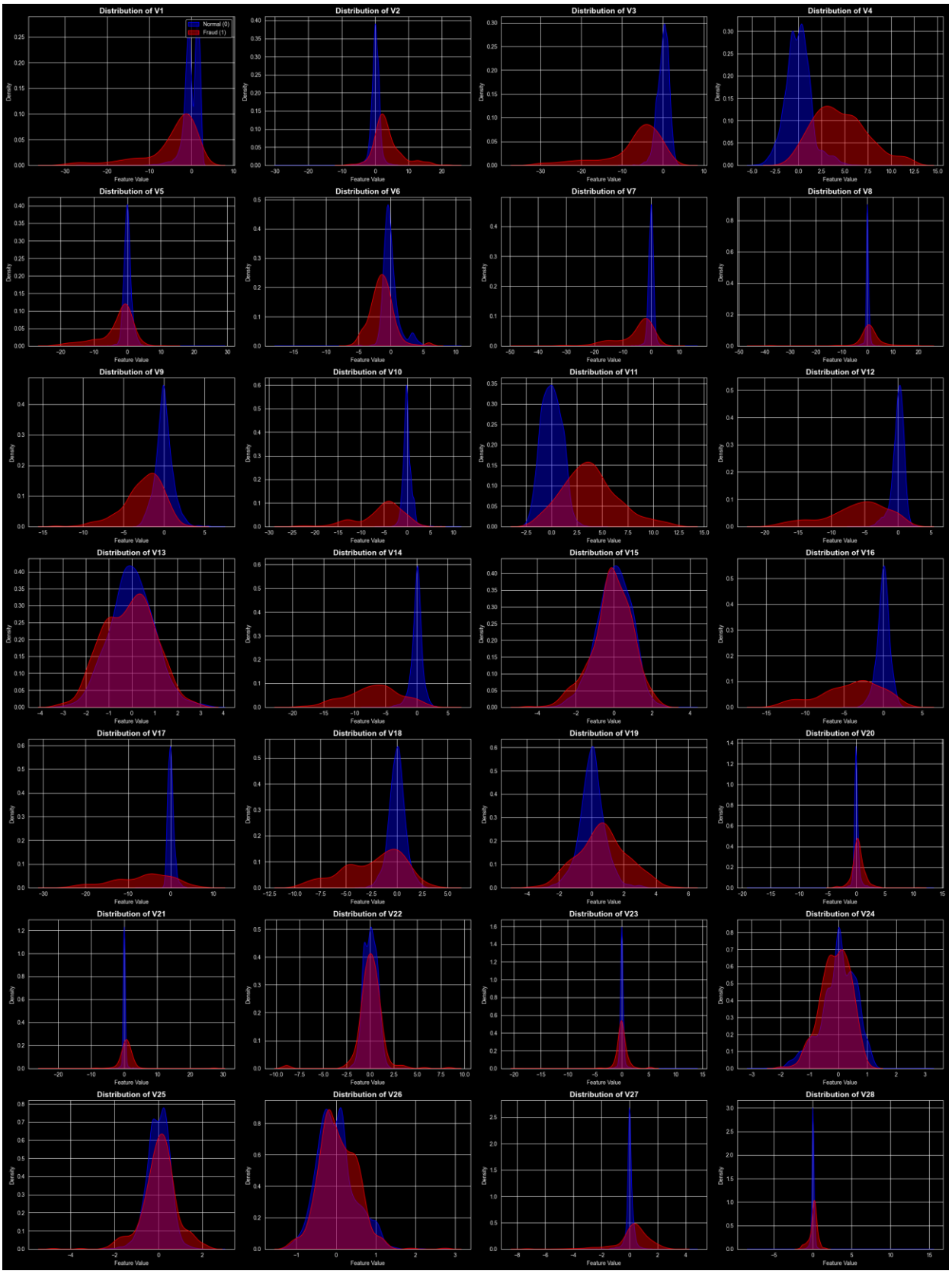
3. EDA & Feature Analysis

Our target variable is Class, where 0 represents a legitimate transaction and 1 represents a fraudulent transaction. This dataset has severe class imbalance, with fraud accounting for approximately 0.17% of all transactions.

The primary input features, V1-V28, are PCA-transformed features derived from original transaction attributes. They are linearly uncorrelated due to PCA, but not necessarily independent. Among all the columns above, we should find the most related features via methods like calculating the correlation with the target variable.

To achieve this, we cross-referenced visual distributions with algorithmic feature importance:

- **Visual Divergence:** We plotted Kernel Density Estimation (KDE) curves comparing fraud vs non-fraud patterns. Dimensions like V12, V14, and V17 displayed severe behavioral divergence—legitimate transactions formed predictable spikes near zero, while fraudulent transactions spread into anomalous, skewed distributions. Conversely, features like V13 and V15 showed massive overlap, acting as mathematical "noise."



- **Final Selection for Linear Baseline:** By removing the overlapping dimensions, we isolated the strongest predictive footprint: **V10, V11, V12, V14, V16, V17**, alongside the scaled Amount.

4. Initial Modeling Pipeline

To ensure our modeling decisions are informed by the data's characteristics and real-world constraints, we built an initial ML pipeline that supports two distinct algorithms with shared preprocessing and evaluation foundations.

- **Input:** Preprocessed feature matrix X (30 columns after scaling) and binary target y (Class).
- **Split:** Stratified 80–20 split to maintain the 0.17% fraud ratio in both sets, utilizing a fixed random_state for reproducibility.

From this shared foundation, the pipeline branches into two specific algorithm workflows:

Branch A: Logistic Regression Pipeline (logistic_regression.ipynb)

1. **Feature Selection:** Dimensionality reduction down to 8 highly predictive features (V14, V12, V17, V11, V10, V16, scaled_amount) identified via correlation and Random Forest importance.
2. **Training:** Fit a LogisticRegression(solver='liblinear', class_weight='balanced', random_state=42) model on the 8 selected features.
3. **Evaluation:** Assess model performance using a classification report, confusion matrix, and Precision-Recall curve, operating at the default decision threshold of 0.5.

Branch B: Random Forest Pipeline (random_forest.ipynb)

1. **Data Balancing:** Apply SMOTE (Synthetic Minority Over-sampling Technique) exclusively on the training set to synthetically balance the classes without causing data leakage.
2. **Training:** Fit a Random Forest classifier on all 30 features to capture complex non-linear interactions. Hyperparameters utilized: n_estimators=200, class_weight={0:1, 1:2}, max_depth=12, and min_samples_leaf=2.
3. **Threshold Tuning:** Implement a vectorized threshold optimization step to strictly enforce a recall ≥ 0.95 , and subsequently maximize precision at that recall level.
4. **Evaluation:** Assess final performance using a classification report,

5. Algorithms Tested and Baseline Results

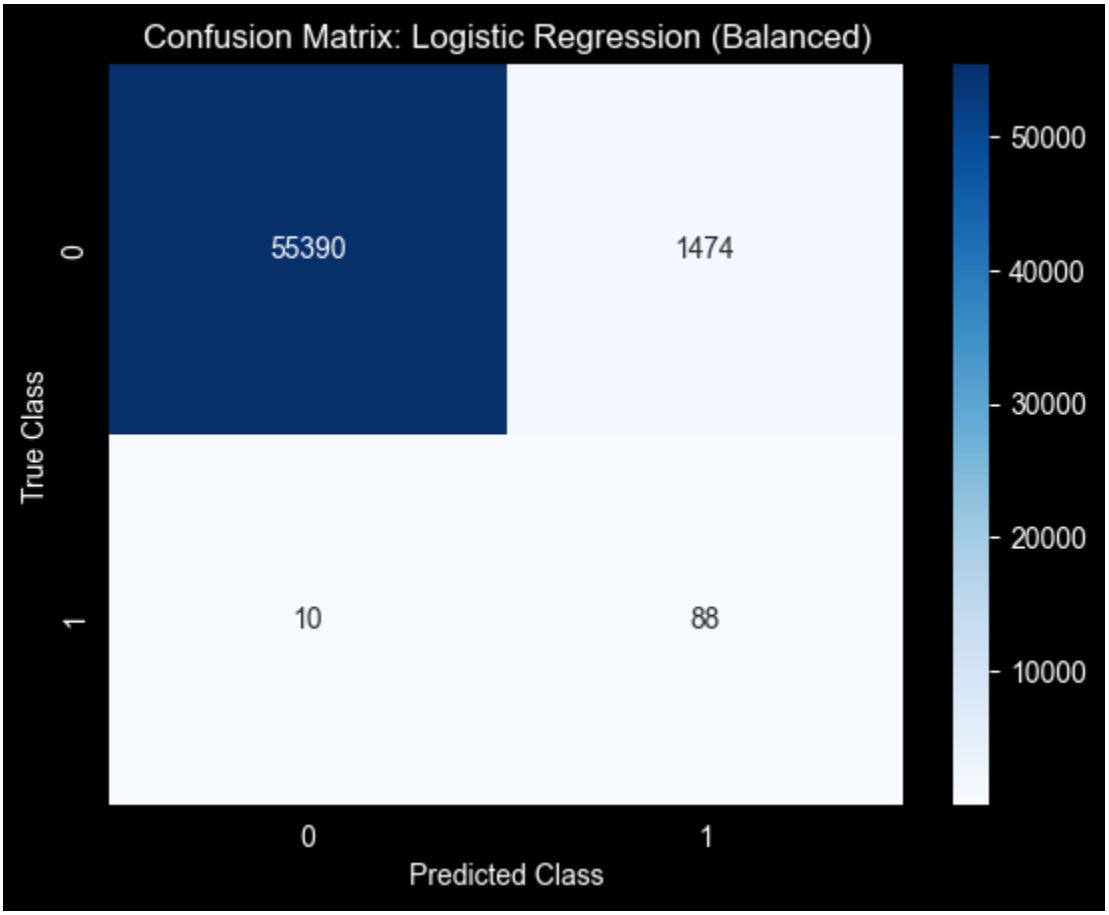
The project requirements mandate that we must experiment with multiple ML algorithms. We defined our baseline models to establish a reference point for later improvements.

Algorithm 1: Binomial Logistic Regression (Baseline)

We utilized a Logistic Regression model configured with the liblinear solver and class_weight='balanced'. Crucially, this linear model was trained **only on the manually selected**

features to reduce noise and prevent the overlapping dimensions from diluting the decision boundary.

- **Early Performance Results:** The model acts as a highly sensitive safety net. It successfully catches the vast majority of actual fraud (High Recall – 90%), satisfying our primary objective. However, operating at the default 0.5 threshold, it suffers from a high rate of false alarms (Low Precision – 6%).



Algorithm 2: Random Forest Classifier (Exploratory)

To test if non-linear feature interactions could naturally improve upon the baseline's false-positive rate, we integrated a Random Forest model. **Notably, unlike our linear baseline, the Random Forest model was trained on all available features.** Because tree-based ensembles inherently perform feature selection by choosing optimal splits, feeding it the full feature space allows us to evaluate if complex, non-linear interactions among the "noisy" features yield better predictive power than our manually reduced dataset.

- **Early Performance Results:** By optimizing the decision threshold, this model demonstrated the ability to strictly maintain a Recall ≥ 0.95 while mapping out the corresponding Precision trade-offs (0.8%).

6. Current Challenges or Obstacles Encountered

• Extreme class imbalance: The extreme ratio (0.17% fraud) continues to heavily influence the decision boundary, necessitating advanced techniques like SMOTE over standard algorithmic weighting.

• Precision/recall trade-off: Minimizing false negatives while controlling false positives remains the central technical hurdle. While the initial models achieve high recall, minimizing the false positive rate to a commercially viable level requires aggressive threshold tuning.

• Threshold selection: The RF threshold is currently tuned on the test set. A better practice is to use a validation set or cross-validation to choose the threshold and reserve the test set for final reporting only.

• Feature selection vs full features: Using 8 features (LR) improves interpretability but may lose signal; using all 30 (RF) preserves signal but reduces interpretability. We have documented both approaches.

• Interpretability: V1–V28 are PCA-derived and not directly interpretable in business terms. Logistic Regression coefficients still indicate which of the 8 features to push toward or away from fraud.

7. Plan for Completion

Remaining tasks

- **Validation-Based Thresholding:** Implement a validation split (or cross-validation) to select the optimal classification threshold for both the Random Forest and Logistic Regression models. This strictly ensures the test set is used *only* for final reporting, preventing data leakage.
- **Document Final Metrics:** Record the final performance metrics (precision, recall, F1-score, and accuracy) for the chosen thresholds. We will explicitly state the mathematical optimization rule used (e.g., "prioritize recall $\ge$ 0.95, then maximize precision for Random Forest").
- **Model Integration & Comparison:** Integrate all planned Logistic Regression improvements (SMOTE, threshold moving, and hyperparameter tuning) and conduct a direct, head-to-head comparison of the final Random Forest versus the final Logistic Regression model on the exact same test set.
- **Notebook Contextualization:** Add a concise write-up within the Jupyter notebooks explaining the precision-recall trade-off and mathematically justifying why we prioritize recall for this specific real-world use case. This ensures strong project continuity with the goals outlined in MLP 1 and the ML Project Document.

Planned improvements or experiments

- **SMOTE for Logistic Regression:** Apply SMOTE exclusively to the training set (mirroring the current Random Forest pipeline architecture) to synthetically balance the classes, and subsequently re-evaluate the linear model's performance.
- **Comprehensive Hyperparameter Tuning:** * *Logistic Regression:* Utilize GridSearchCV to optimally tune the regularization parameter C.
- *Random Forest:* Tune n_estimators, max_depth, min_samples_leaf, and class_weight using the validation set.
- **Threshold Moving Strategy:** For both models, systematically experiment with shifting the decision thresholds (e.g., optimizing specifically for the F1-score or meeting a strict recall target). We will compare these strategies graphically and quantitatively using the Area Under the Precision-Recall Curve (AUPRC).
- **Random Forest Precision Enhancements:** Experiment with different SMOTE sampling ratios and RF hyperparameters to investigate if precision can be incrementally improved while strictly maintaining our safety-net constraint of recall ≥ 0.95 .

Timeline or next steps toward project completion

- **Near Term:** Implement validation-based threshold selection across the codebase and update the notebooks so the test set is securely reserved for final metrics only.
- **Next Phase:** Run the Logistic Regression pipeline incorporating SMOTE and threshold tuning. Record the resulting metrics and add the updated performance comparison to this report's framework.
- **Final Phase:** Run the Random Forest model utilizing the new threshold selection methodology. Document the final numbers and comprehensively compare Random Forest vs. Logistic Regression to generate the insights for the final written report and repository deliverables.

8. GitHub Repository and Notebooks

Our current work and progress are demonstrated in our project Git repository (Hongru, Ramya) and in the following notebooks:

• random_forest.ipynb — Random Forest pipeline: preprocessing, SMOTE, training, recall-first threshold tuning, evaluation, challenges, and plan.

• logistic_regression.ipynb — Logistic Regression pipeline: preprocessing (RobustScaler), feature selection (correlation + RF importance), baseline model, classification report, confusion matrix, Precision-Recall curve, scenarios discussion, challenges, and plan.

These deliverables align with the MLP2 requirements: data preprocessing, EDA summary, initial ML pipeline, baseline models and early results, feature analysis, algorithms tested, challenges, and Plan for Completion.